

Übung B – Self-Signed Certificate für einen WebServer erstellen

- Environment
Diese Übung ist für **Windows** ausgelegt – also den eigenen **Laptop** verwenden!
OpenSSL für Windows: <https://indy.fulgan.com/SSL/> (letzte Version 1.0.2u)
- Ziele
 - Selbst eine **Root-CA** erstellen
 - Ein **Zertifikat** für einen **WebServer** erstellen und von der eigenen CA signieren
(Der **Schülername** muss im Zertifikat aufscheinen - CN, Email)
 - Root-CA in den **Zertifikatsspeicher** legen
 - Einen lokalen WebServer mit **HTTPS** und Zertifikat aktivieren und testen
- Help
PDF-Anleitung: „**How to Create Trusted Self-Signed SSL Certificates and Local Domains for Testing.pdf**“

Achtung! Prinzipiell nach der PDF-Anleitung vorgehen – aber die nachfolgenden Informationen beachten (vor allem jene in Rot) – besonders wenn ein Befehl nicht funktioniert!

Task 1: Eigene (lokale) Root-CA erstellen

- a) Lokale Domännennamen im **hosts**-file anlegen (zum Testen) – PDF-Anleitung Seite 3
- b) **Private Key** für die **Root-CA** erstellen (rootCA.key) – PDF-Anleitung Seite 4
Dabei muss ein Passwort erstellt werden: **Labor4**
- c) Ein **Root-Zertifikat** erstellen – der Befehl in der Anleitung funktioniert nur wenn ein config-file angegeben wird. Also `-config rootCA.cnf` anhängen.

Zuvor muss allerdings das config-file im lokalen Ordner erstellt werden.

Inhalt von `rootCA.cnf`:

```
[req]
default_bits = 2048
prompt = no
default_md = sha256
distinguished_name = dn

[dn]
C=AT
ST=Niederoesterreich
L=Wiener Neustadt
O=HTLWRN
OU=Informatik
emailAddress=admin@example.com
CN = SCHUELERNAME's Root Certificate
```

- d) Nun das erstellte Zertifikat in den **Zertifikatspeicher von Windows** importieren – PDF-Anleitung Seite 5
- e) Den Zertifikatspeicher von Windows öffnen und das Zertifikat anzeigen
 - Optional: Das **Zertifikat im DER-Format exportieren** und den Inhalt mit dem ursprünglichen Zertifikat (BASE64-Format) vergleichen

- f) Das eigene Root-Zertifikat in den verwendeten **Browser** importieren (Chrome, Firefox,...) – PDF-Anleitung Seite 6

Task 2: SSL-Zertifikat erstellen für einen lokalen WebServer

Es muss ein **Zertifikats-Request** erstellt und dieser dann **mit der root-CA signiert** werden.

– PDF-Anleitung Seite 7

- g) Konfig-File für das Zertifikat anlegen: `server.csr.cnf`: (**Email anpassen!**)

```
[req]
default_bits = 2048
prompt = no
default_md = sha256
distinguished_name = dn

[dn]
C=AT
ST=Niederoesterreich
L=Wiener Neustadt
O=HTLWRN
OU=Informatik
emailAddress=SCHUELERNAME@example.com
CN = SCHUELERNAME
```

- h) Extra-File für die Domännennamen erstellen `v3.ext`

```
authorityKeyIdentifier=keyid,issuer
basicConstraints=CA:FALSE
keyUsage = digitalSignature, nonRepudiation, keyEncipherment,
dataEncipherment
subjectAltName = @alt_names

[alt_names]
DNS.1=myexample.com
DNS.2=sub.myexample.com
DNS.3=myexample1.com
DNS.4=localhost
```

- i) Zertifikatsanfrage erstellen (certificate-signing request – **CSR**) - PDF-Anleitung Seite 8

Daten aus dem config-file laden!

Output: `server.csr` + `server.key`

- j) Der **CSR** kann danach **mit dem Root-Zertifikat signiert** werden

Das erstellte Zertifikat für unseren Server heißt **`server.crt`**

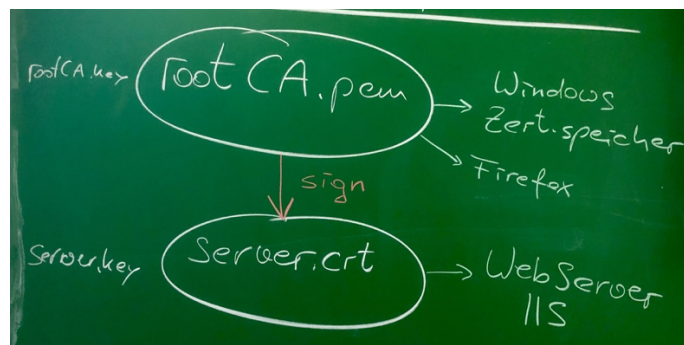


Abbildung 1: Root- und Server-Zertifikat

Task 3: SSL-Zertifikat in einem WebServer testen

- k) Wer einen **lokalen WebServer** laufen hat kann versuchen das Zertifikat dort zu hinterlegen und HTTPS auf dem WebServer zu aktivieren.
In der PDF-Anleitung wird ein temporärer Server mit JavaScript erstellt und mit dem Zertifikat gestartet.
- l) Alternativ kann versucht werden einen **temporären Web-Server mit openssl** zu starten: <https://www.madboa.com/geek/openssl/#how-do-i-set-up-an-ssl-server-from-the-command-line>

```
# the -www option will sent back an HTML-formatted status page  
# to any HTTP clients that request a page  
openssl s_server -cert mycert.pem -www
```

```
# the -WWW option "emulates a simple web server. Pages will be  
# resolved relative to the current directory." This example  
# is listening on the https port, rather than the default  
# port 4433  
openssl s_server -accept 443 -cert mycert.pem -WWW
```

Überprüfung:

- Mit einem **Browser** über **HTTPS** auf den WebServer zugreifen mit
 - <https://localhost>
 - und
 - <https://myexample.com>
- Es sollte **keine Fehlermeldung im Browser** erscheinen! (weil das Zertifikat der Root-CA im lokalen Speicher liegt)
- Das **Zertifikat** der Webseite im Browser ansehen und die persönlichen Informationen finden (**Name**, E-Mail)
- Das eigene **Root-CA** im **Zertifikatsspeicher** zeigen